HACKATON M1 AL IABD · 21 AVRIL 2026

いただき

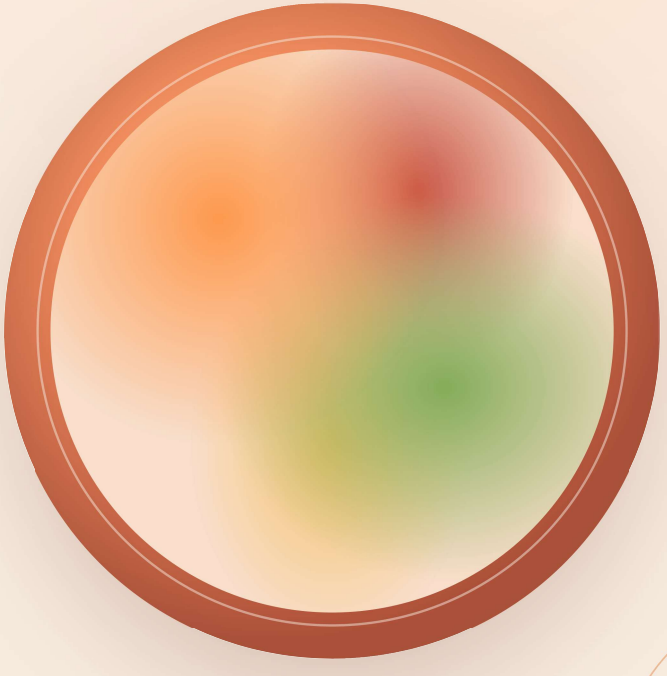
Du *Code* à l'Assiette

POC d'application de tracking nutritionnel propulsée par une IA locale multimodale.

ÉQUIPE
Faloci · Hugo · Letaief

ÉCOLE
ESGI — NotebookLM

VERSION
POC v0.1



CHAPITRE 01

L'équipe *& la mission*

Trois rôles, une ambition : prouver que l'IA multimodale peut tourner localement et servir une app nutritionnelle de bout en bout — d'une seule codebase.

LEAD IA & PROMPTS

01

Lorenzo Faloci

Stratégie IA, design des prompts, orchestration de l'infrastructure d'inférence.

Prompt design · Infra

IA LOCALE · GPU

02

Hugo Derniaux

Infra IA locale sur poste GPU — Ollama, gestion VRAM, exposition via Ngrok.

Ollama · GPU · Ngrok

BACKEND & ARCHITECTURE

03

Ahmed Letaief

Socle Spring Boot, architecture en couches, modèle de données et sécurité JWT.

Spring Boot · Architecture

Une codebase unique · Trois expertises convergentes

CHAPITRE 02

Parcours *utilisateur*

i.

Inscription & connexion

Sécurisation par **JWT**, session stateless.

ii.

Téléversement photo

L'utilisateur prend en photo son assiette depuis mobile ou web.

iii.

Analyse automatique

Identification du plat + estimation calorique via LLM multimodal.

iv.

Correction / relance

« Ce n'est pas du poulet, c'est du tofu » — l'utilisateur affine, l'IA recalcule.

v.

Historique 7 jours

Repas, analyses et calories ingérées en graphique.

Boucle de correction — l'utilisateur peut relancer l'analyse à tout moment.

CHAPITRE 03

L'arsenal *technique*

FONDATION

Java 25 LTS + Spring Boot

4.0.5

Génération start.spring.io — pile ultra-récente, support long terme.

INTELLIGENCE

Spring AI + Ollama

2.0.0-M4

LLM multimodal local — pas de dépendance cloud, coût nul par inférence.

PERSISTANCE

Spring Data JPA + Base H2

embarquée

Stockage éphémère optimisé pour itération rapide pendant le hackaton.

SÉCURITÉ & API

Spring Security JWT + Swagger

7 · springdoc 3.0.2

Protection & documentation — auth stateless, contrat d'API auto-généré.

CHAPITRE 04

Architecture complète *& workflow IA*

PARTIE ARCHITECTURE — AHMED

Architecture en couches

VUE	Interface utilisateur · templates
CONTRÔLEUR	Endpoints REST · DTO
SERVICE	Logique métier · orchestration IA
REPOSITORY	Accès aux données · JPA
PERSISTANCE	Base H2 embarquée
MODEL	Entités · Domain

PARTIE IA — HUGO & LORENZO

Pipeline d'inférence

Photo (Entrée) → Spring AI (Gestion) → Ollama (Moteur local) → LLM Multimodal (qwen2.5vl:7b) → JSON (Sortie)

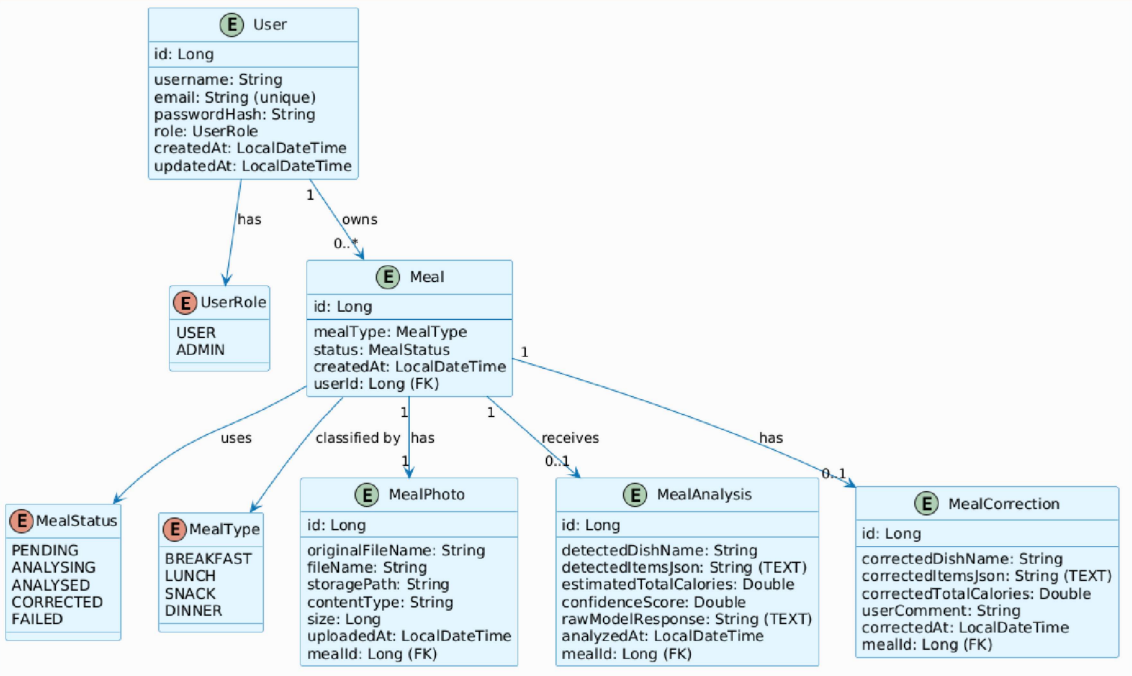
Machine hôte
PC de Hugo

CPU	RAM	GPU
AMD Ryzen 9 HX	32 Go	NVIDIA RTX 5060

CHAPITRE 05 · AL — AHMED

Diagramme *de classes*

Modèle de données du domaine : utilisateur, repas, analyse IA et ingrédients — reliés par les services et exposés via les contrôleurs REST.



Entités *clés*

`User` · `Meal` · `Analysis` · `Ingredient` — liens 1..N via JPA, identifiants UUID, timestamps auto.

Séparation *claire*

Les DTO isolent la couche REST des entités — chaque mutation passe par le service, jamais directement depuis un contrôleur vers le repository.

Mapping *MapStruct*

Conversion entité ↔ DTO générée à la compilation, zero réflexion runtime, tests garantis sur chaque champ.

CHAPITRE 06 · AL — AHMED

Diagramme *de séquence*

Trace l'appel complet d'une analyse — depuis la photo envoyée par l'utilisateur jusqu'au JSON persisté en base, à travers chaque couche de l'architecture.

Diagramme détaillé ouvert en externe

Le diagramme de séquence complet est présenté à l'écran séparément pendant la soutenance — lisibilité optimale sur un second document pour éviter la surcharge visuelle des slides.

diagramme-sequence.pdf

i.
Client → Contrôleur
POST /api/meals — multipart + JWT

ii.
Contrôleur → Service
AnalysisService.analyze(file, user)

iii.
Service → Ollama
Appel HTTPS via RestClient + tunnel

iv.
Service → Repository
Persist Analysis + Ingredients

CHAPITRE 07 · INFRASTRUCTURE

Le problème *d'accès local*

PC démonstrateur
AHMED
IP 10.213.0.x

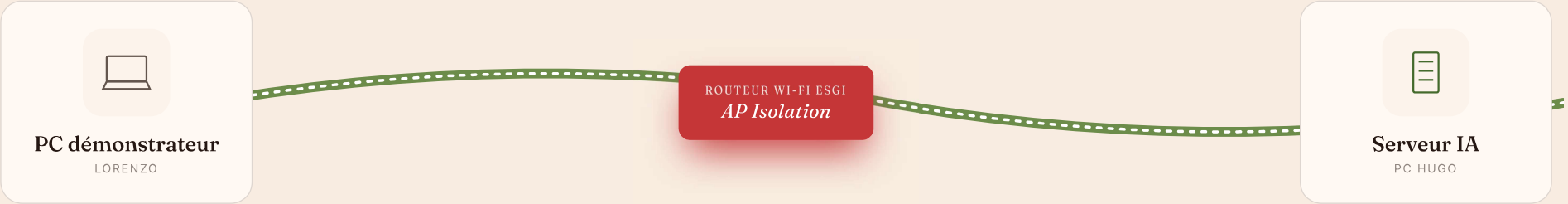
Serveur IA
PC HUGO
IP 10.213.203.128 · :11434

Problème
Le PC de Hugo n'est **pas accessible** depuis le réseau Wi-Fi maison sur le port 11434.

Piste de solution
Mise en place d'un **tunnel Ngrok** pour exposer l'instance Ollama locale en HTTPS.

CHAPITRE 08 · INFRASTRUCTURE

Le contournement : *Tunnel Ngrok*



Endpoint public : `https://ducky-shrank-washer.ngrok-free.dev`

Symptôme

Ping et TCP `:11434` bloqués. Le routeur empêche le trafic client-à-client sur le LAN.

Injection d'en-tête requise

Via `@Bean RestClient` — header `ngrok-skip-browser-warning: any` pour contourner l'intercepteur HTML.

CHAPITRE 10

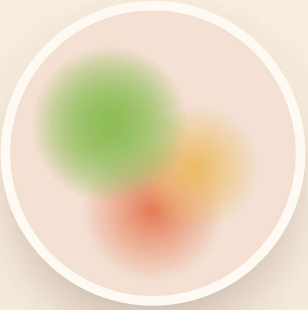
Le choix du *moteur IA*

	CHOIX PRINCIPAL ✦ qwen2.5vl:7b	FALLBACK gemma3:4b	PERSPECTIVE SCALABILITÉ Modèles Cloud
Poids / VRAM	**6 Go**	3,3 Go	N/A
Langue native	**Excellent (FR natif)**	À tester	Excellent
Format JSON strict	**Parfait du 1er coup**	Inconnu	Parfait
Multimodal	**Oui · analyse pixels**	Oui	Oui

Qwen2.5vl retenu — détecte jusqu'à 6 ingrédients par photo avec une structure JSON infaillible en français.

CHAPITRE 11

Pipeline *de transformation multimodale*



Pixels non-structurés

→

Spring AI + Ollama

```
temperature = 0.2
format      = json
num_ctx     = 4096
language    = "français"
schema      = {nomPlat, ingredients[]}
```

→

```
{
"nomPlat": "...",
"caloriesMin": 480,
"caloriesMax": 620,
"ingredients": [...]
}
```

JSON strict

! **Gestion sémantique des portions** — petit / moyen / grand — pour éviter l'hallucination du LLM sur les grammages exacts.

CHAPITRE 12

Le moteur Itadaki *en action*

i. **Upload photo** `/api/meals`

→

ii. **Validation JWT** `SecurityFilter`

→

iii. **AnalysisServiceImpl** `orchestre`

→

iv. **OllamaRestClient** `+ tunnel`

→

v. **Ollama local** `qwen2.5vl`

→

vi. **Réponse JSON strict** `parse`

→

vii. **Persistance H2** `MapStruct`

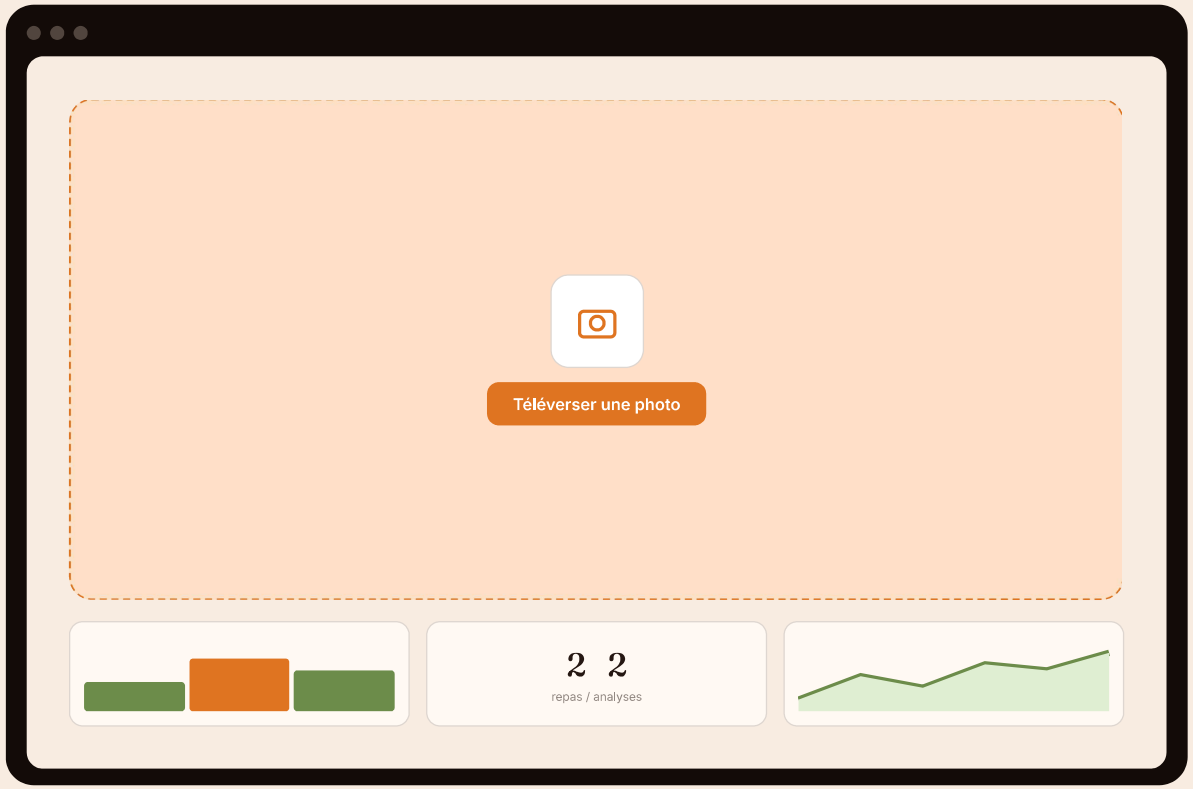
Entrée
`multipart/form-data + JWT`

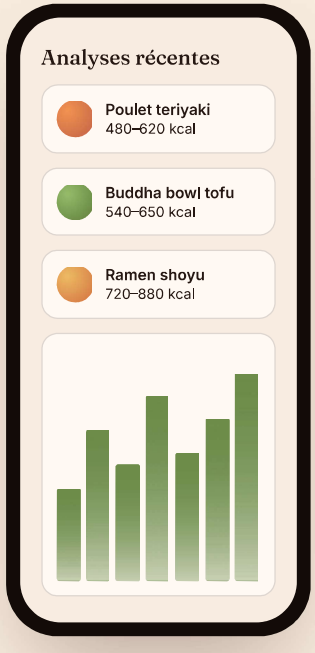
Réseau
`HTTPS · tunnel ngrok`

Sortie
`Analysis{...} sauvée`

CHAPITRE 13 · LIVE

Démonstration *du POC Itadaki*





i. Inscription & auth

ii. Analyse d'une assiette en direct

iii. Correction sémantique (2e passe)

iv. Historique des consommations

CHAPITRE 14

Rétrospective *des performances*

~50*s*

Cold start

Chargement initial du modèle en VRAM — `keep-alive` de 5 min pour éviter les rechargements successifs.

1,65*s*

Analyse JSON simple

Extraction rapide des entités (nom du plat, ingrédients, fourchette calorique) via Qwen une fois le modèle chaud.

9,7*s*

Correction stateless (2e passe)

Recalcul sémantique (ex. poulet → tofu) et réajustement calorique `480–620 → 450–580 kcal`.

Performance à chaud parfaitement adaptée pour une utilisation applicative fluide.

CHAPITRE 15

Perspectives *& évolutions*

Gamification
Objectifs caloriques journaliers personnalisés, streaks et badges hebdomadaires.

Précision nutritionnelle
Intégration de la base **CIQUAL** pour des calculs stricts *hors-LLM*, avec le LLM réservé à la détection visuelle.

Suivi de tendance
Analyse de tendance macro-nutriments sur plusieurs semaines, recommandations contextualisées.

> *Itadaki prouve qu'avec une **architecture résiliente**, l'IA générative locale peut transformer notre rapport à la nutrition — du code jusqu'à l'assiette.*
>
> — *Équipe Itadaki · Hackaton M1 AL IABD*